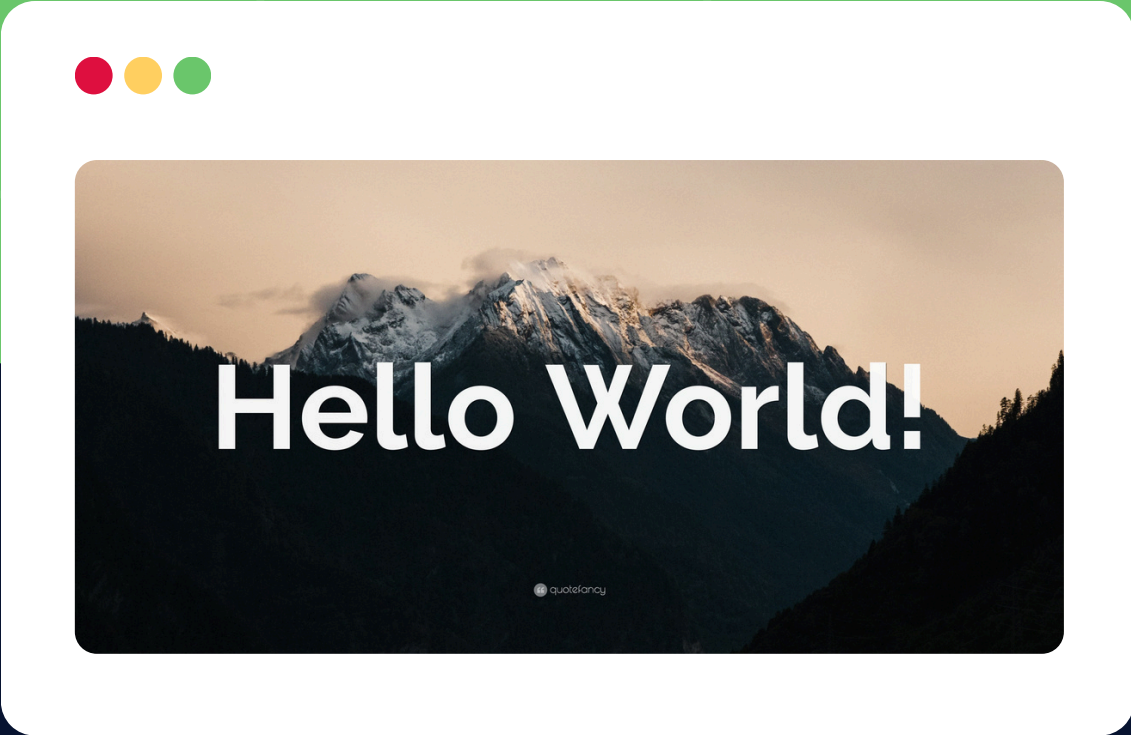
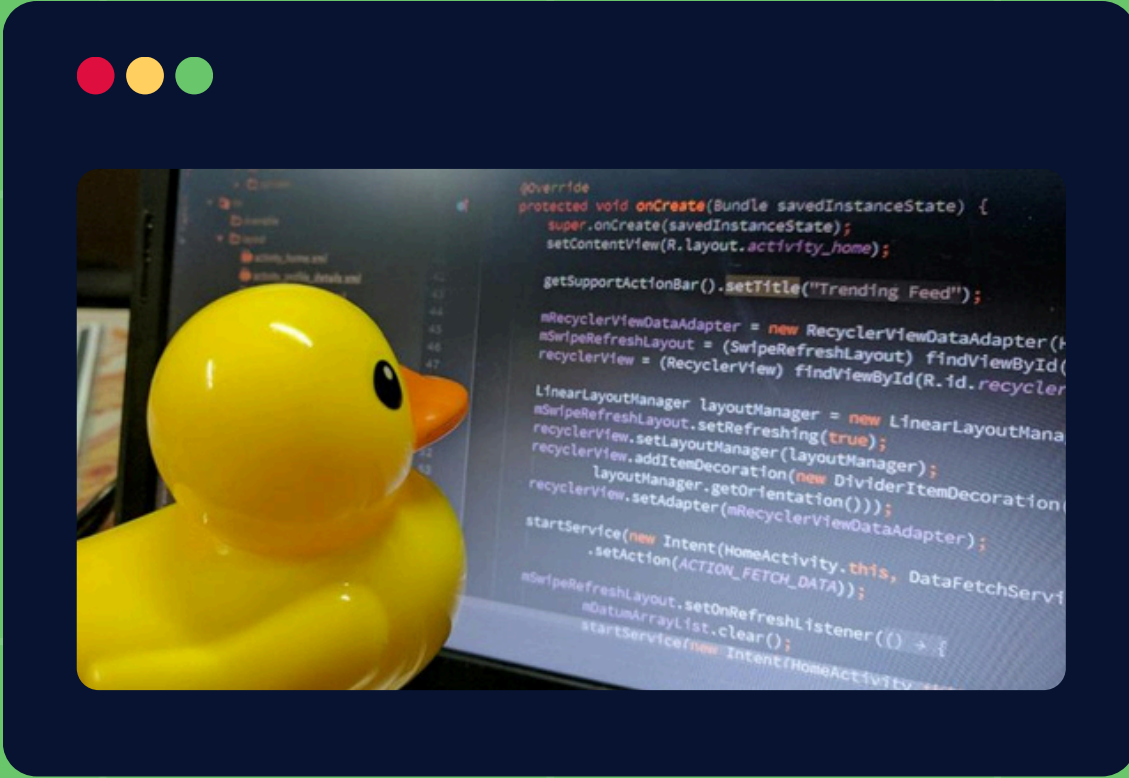




Hello World!



Model Selection

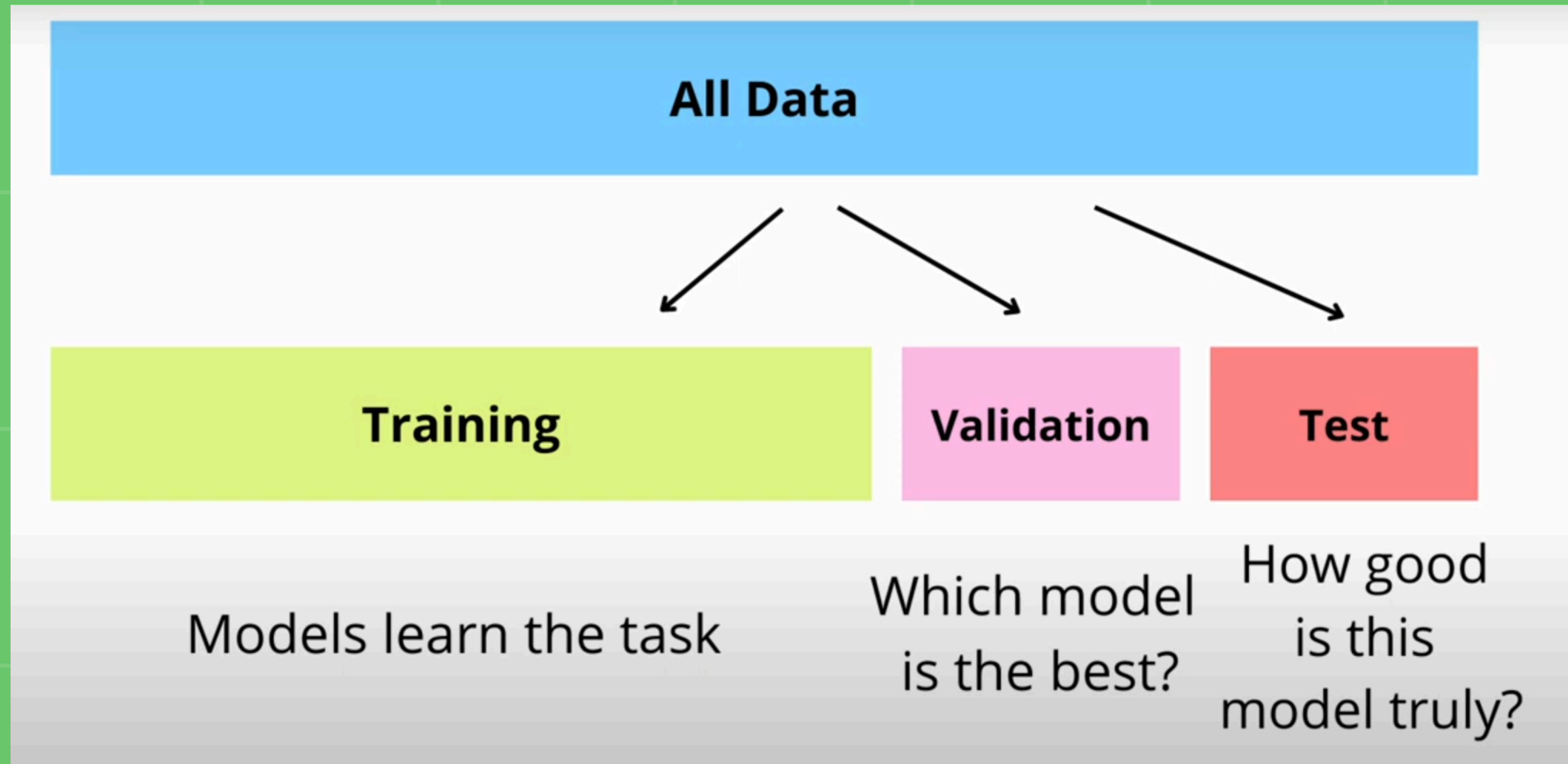


Session -> 13



Train- Validation- Test

Why We Need Validation set?



Why We Need Validation set?

1- What if the test set is too easy?

- Test accuracy might be artificially high, higher than the train itself, giving a false sense of performance.
- Without a validation set, you don't have a second opinion during training.

2- "Is the test set enough to choose the best model?"

- Not really, because if you try multiple models and evaluate them all on the test set, you're picking what performs best on that specific test set → not what generalizes best.

Why We Need Validation set?

3- What if there's a data leak between train and test?

- You'll get very high accuracy, but the model didn't really learn, it memorized leaked information.
- This happens when: Duplicate or overlapping data exists in train/test - Preprocessing is done before splitting.

A validation set gives you an additional chance to detect suspicious patterns before you ever touch the test set.

How to use it?

Step 1: split into train+val and test

```
X_temp, X_test, y_temp, y_test = train_test_split(X, y, test_size=0.1, stratify=y, random_state=42)
```

Step 2: split train+val into train and validation

```
X_train, X_val, y_train, y_val = train_test_split(X_temp, y_temp, test_size=0.1111, stratify=y_temp, random_state=42)
```

Detect the model's Problems

Case 1 : Underfitting

- Train Accuracy: Low 
- Validation Accuracy: low 
- Test Accuracy: Low 

Solutions → Choose a more complex model, or a model that fits the data better .

(example: choose polynomial regression instead of linear if the data is not linear or choose polynomial kernel in SVM)

Detect the model's Problems

Case 2 : Overfitting

- Train Accuracy: High 
- Validation Accuracy: low 
- Test Accuracy: Low 

Solutions → Reduce model complexity, or use methods to overcome overfitting such as regularization.

Detect the model's Problems

Case 3 : Failed to generalize

- Train Accuracy: High 
- Validation Accuracy: High 
- Test Accuracy: Low 

Here, the test may contain data that did not appear in the training and validation, so it works in validation but will fail in real life.

Solutions → Fix how the dataset was split, gather more data, or change the dataset.

Detect the model's Problems

Case 4 : Data Leakage

- Train Accuracy: High 
- Validation Accuracy: High 
- Test Accuracy: Low 

Solutions → Search for the reason of the data leak, such as preprocessing or feature engineering before splitting, drop duplicates.

Excercise

- Which one is the best model?

Model	Train	Validation	Test
A	98%	94%	70%
B	75%	74%	73%
C	99%	84%	83%
D	85%	84%	83%

Conclusion

The best model isn't the one with the highest test score; it's the one with the balanced and consistent accuracy across training, validation, and test sets.

Recap

Dataset	Purpose	What it tells us
Training Set	To learn patterns and fit model parameters (weights)	Did the model learn the data?
Validation Set	To tune hyperparameters and select models	Did we find the best model setup? Does it generalize well?
Test Set	To evaluate final performance on unseen data	Did we succeed? How will it perform in the real world?



Cross Validation

What Is Cross Validation?

- Say that we have 3 models for classification, we will choose the one with the best train and test accuracies, right?
- We may use a 75:25 train-test split or an 80:10:10 train-validation-test split. **How do we know that the data was split in a fair and representative way?**
- Since the split is done randomly, there's a chance that:
 - The training set might not include all important patterns
 - The test set might be too easy
 - The class distribution may be uneven.

What Is Cross Validation?

Train Test	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5
	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5

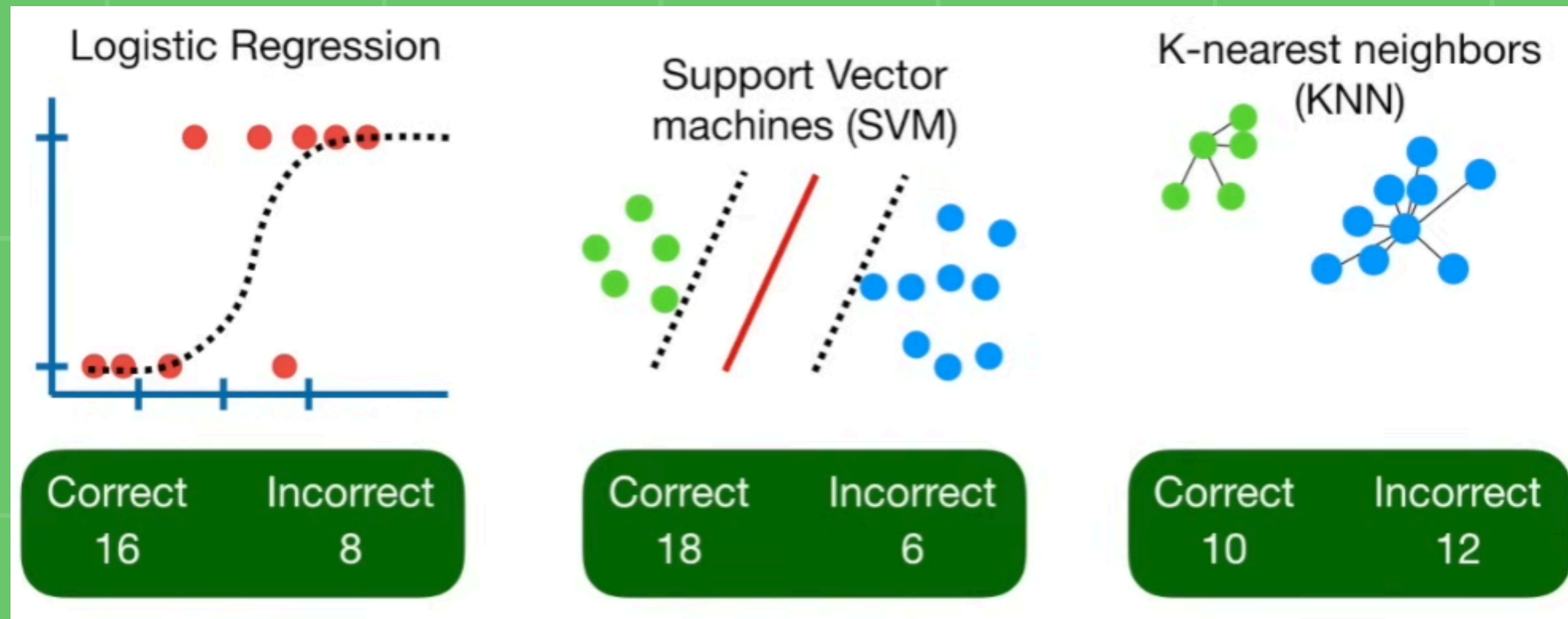
Instead of relying on just one train-test split, cross-validation splits the data into multiple parts (called "folds") and rotates which part is used as the test set. This way, every data point gets a chance to be in both the training and test sets.

What Is Cross Validation?

- **Cross-validation** is a technique used to evaluate the performance of a machine learning model by dividing the dataset into multiple parts (**called folds**) and systematically rotating which part is used for testing. This helps ensure that the evaluation is not dependent on a single train-test split and gives a more reliable estimate of the model's performance on unseen data.

Example

- If we tried a cross-validation of 22 folds, and these are the results of our models, we choose the model that gives us the best results in most folds:



How to use it:



```
from sklearn.datasets import load_iris
from sklearn.model_selection import cross_val_score
from sklearn.tree import DecisionTreeClassifier
```

```
X, y = load_iris(return_X_y=True)
```

```
model = DecisionTreeClassifier()
```

```
scores = cross_val_score(model, X, y, cv=5)
```

```
print("Cross-validation scores for each fold:", scores)
```

```
print("Average accuracy:", scores.mean())
```



Overcome Overfitting

Choosing the Right Model

- We compare multiple models: some may underfit, some may overfit, and at least one achieves a good balance. In this case, **we choose the balanced.**

- **But what if none of our models are well-balanced?**

Suppose we only have one model that underfits and several that overfit. What to do?

Solution → choose the overfitting model with the best potential, and then apply techniques to reduce overfitting and improve generalization.

Ways To Overcome Overfitting

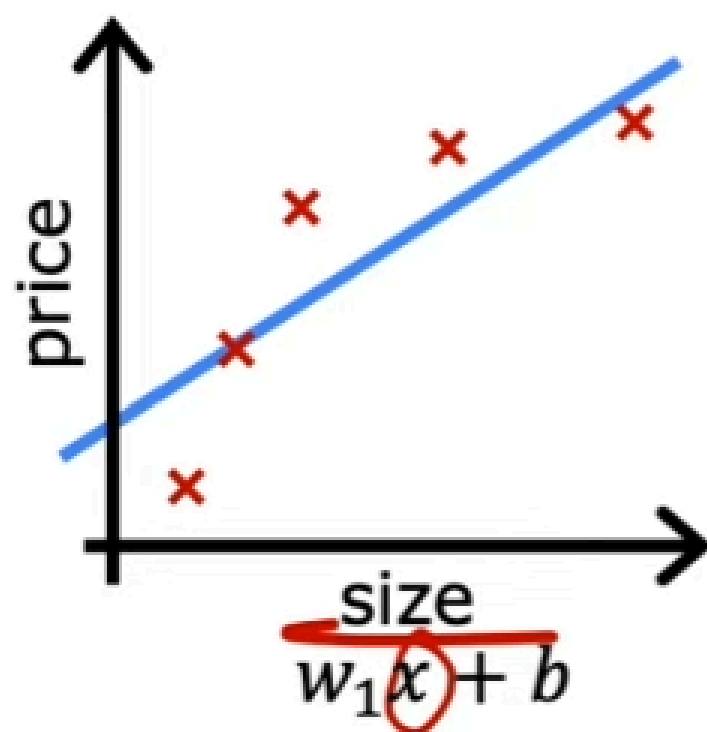
- Gather more data
- Feature Selection
- Regularization

What is Regularization?

- Regularization is a technique used in machine learning to **prevent a model from overfitting** the training data. Overfitting happens when a model learns the noise or random fluctuations in the training data, rather than the actual underlying patterns.
- Regularization works by adding a **penalty** to the model's loss function, it discourages the model from learning overly complex patterns or assigning too much importance (too large weights) to any one feature.

What is Regularization?

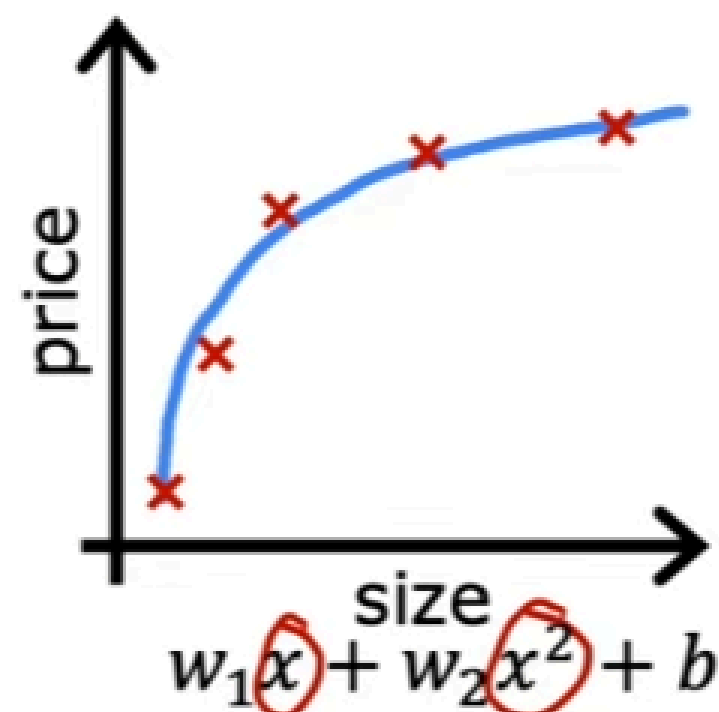
Regression example



underfit

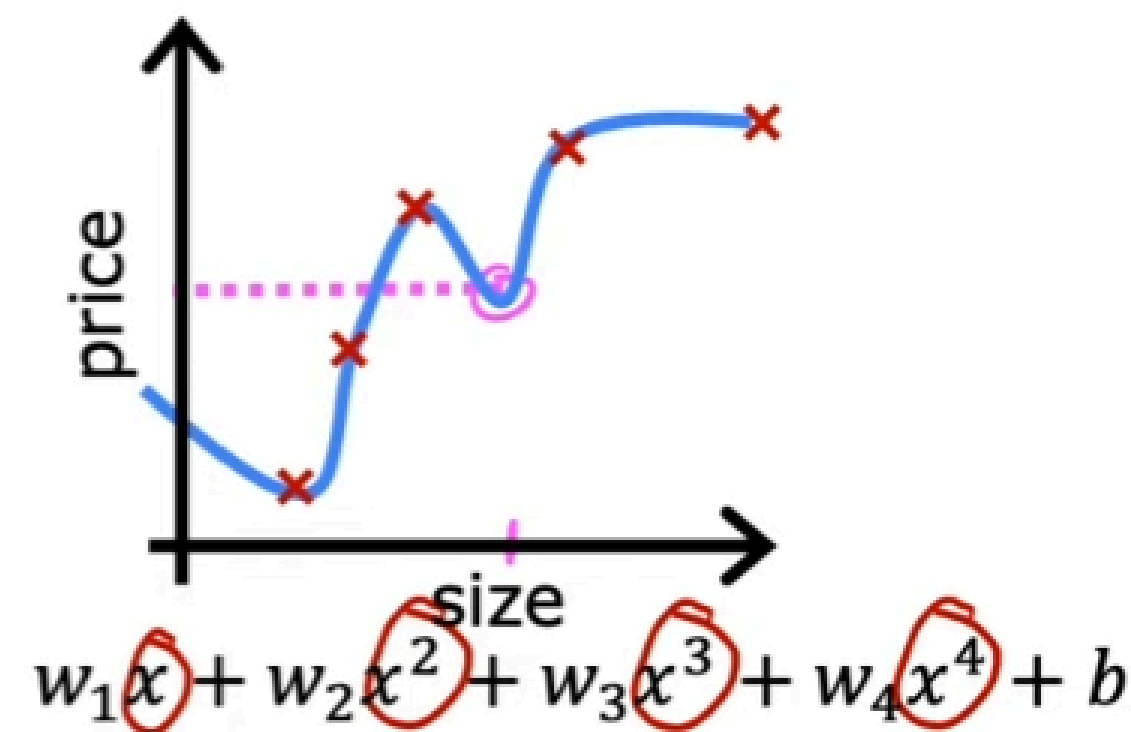
- Does not fit the training set well

high bias



- Fits training set pretty well

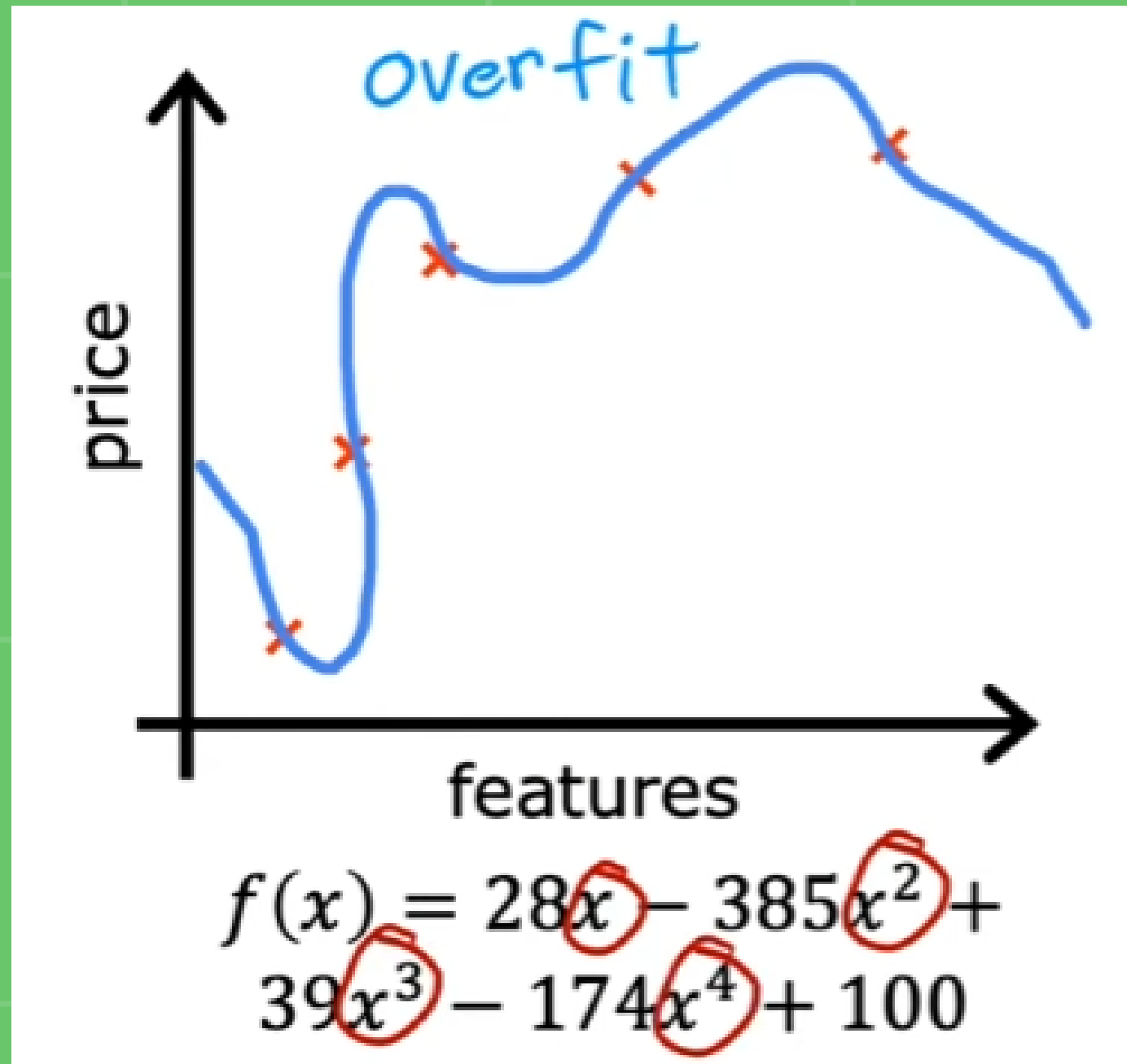
generalization



overfit

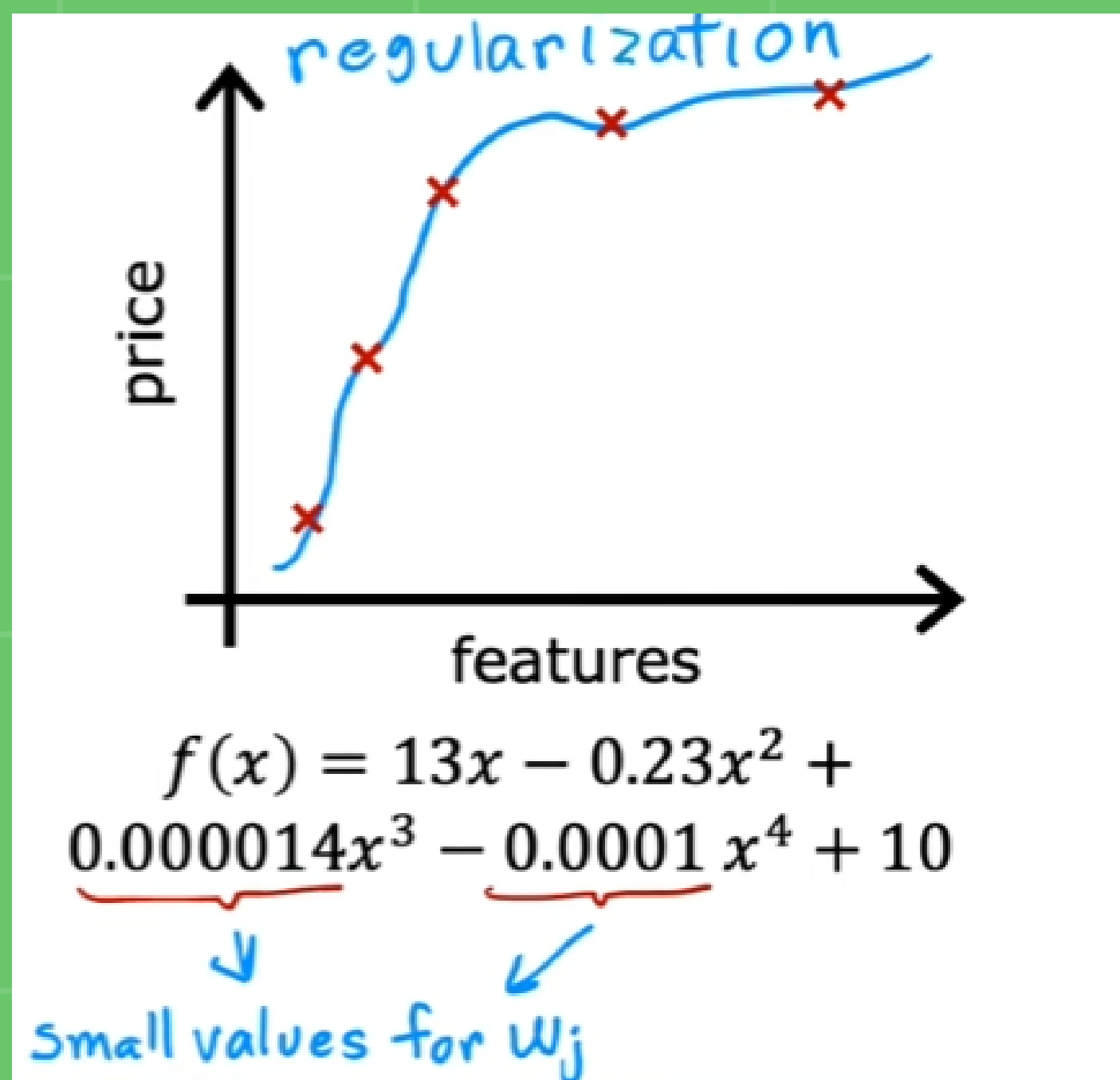
- Fits the training set extremely well

What is Regularization?



- Here we can see that the model is too complex for the data, and it causes overfitting.
- The model is learning complex patterns recognized in high-degree polynomials, which allow the model to bend too much to fit the data points perfectly."

What is Regularization?



- Instead of eliminating the features, Regularization helps us to make these features with a lower impact by adding a penalty in the cost function.

Types of Regularization

- Ridge Regularization
- Lasso Regularization

Ridge Regularization (L2)

$$L2 = ||B||^2 = B_1^2 + B_2^2 + \dots + B_n^2$$

$$RSS_{L2} = \sum_{i=1}^n (Y_i - \hat{Y}_l)^2 + \lambda \sum_{j=1}^P B_j^2$$

- This penalty is controlled by λ (lambda), a tuning factor.
- The penalty increases when the model's weights become large. This encourages the model to keep the weights small, which helps reduce complexity and avoid overfitting.

Lasso Regularization (L1)

$$L_1 = ||B||_1 = B_1 + B_2 + \dots + B_n$$

$$RSS_{L2} = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 + \lambda \sum_{j=1}^p |B_j|$$

- This penalty is controlled by λ (lambda), a tuning factor.
- The penalty increases when the **absolute values** of the model's weights become large. As a result, the model is encouraged to reduce some weights to zero, effectively removing less important features.

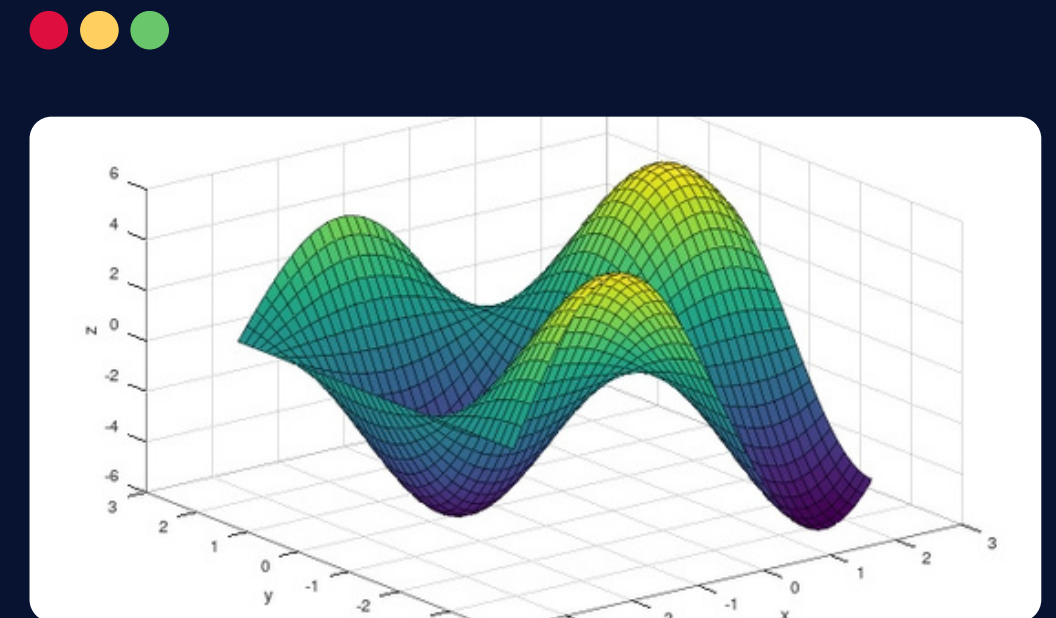
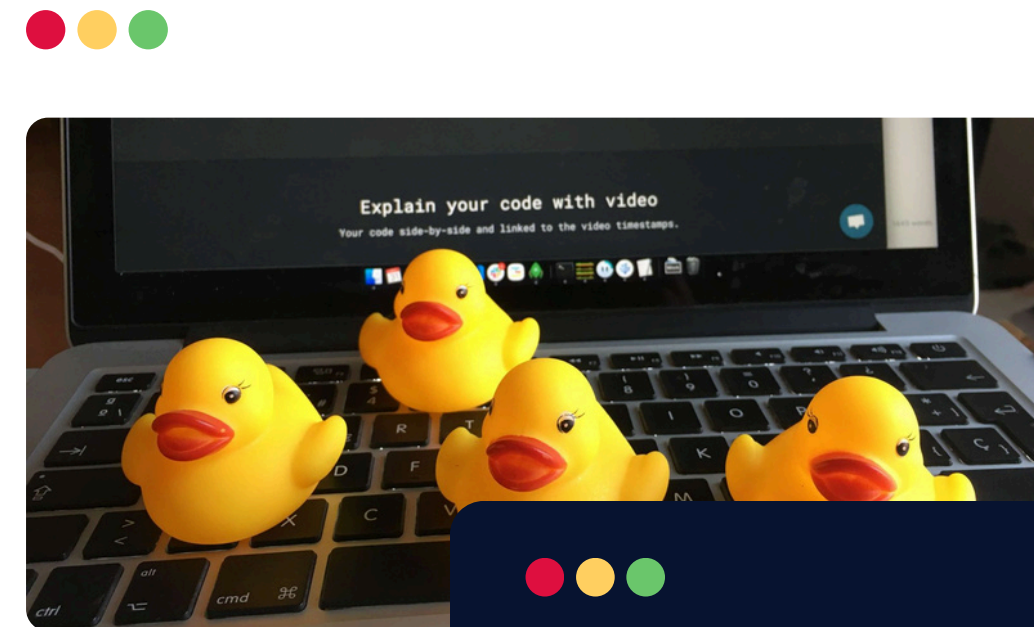
When To Use Which?

Ridge

Lot of features
In the dataset and
all features have
small coefficients.

Lasso

Small number of
features and few
features have high
coefficient value.



Thank You

created by:
Ahd Ehab